

OPTIMISATION DU CODE INTERMÉDIAIRE

Optimisation du code intermédiaire

- Une **étape clé** du processus de compilation visant à améliorer l'efficacité du programme sans en modifier la sémantique.
- Elle intervient après la génération du code intermédiaire et avant la génération du code machine.
- Son objectif est d'**optimiser l'exécution** du code en le rendant plus rapide et moins consommateur de ressources, **sans en altérer les fonctionnalités**.

Parmi ces opérations, on trouve:

- Propagation de copie
- Propagation d'expressions
- Élimination des expressions redondantes
- Simplification algébrique
- Élimination de code inutile

1. Propagation de copie

- Une technique qui consiste à **remplacer** une variable **par** sa valeur ou une autre variable à laquelle elle a été affectée.
- Cela permet d'**éliminer** les affectations inutiles et d'améliorer l'efficacité du code.
- **Principe:**

Lorsqu'une variable x est affectée à une autre variable y ($y = x$), on peut directement remplacer y par x dans le reste du programme (**si aucune autre affectation ne modifie x ou y entre-temps**).

Exemple:

Si deux variables sont égales on remplace une par l'autre afin de l'éliminer après.

Avant optimisation:

```
t1= t2 ;  
t3= 4* t2 ;
```

Après optimisation:

```
t3= 4* t1 ;
```

Cas où la propagation n'est pas possible:

Elle n'est pas possible si la variable affectée change avant d'être utilisée.

Exemple:

```
t1= t2 ;
```

```
t1= t3;
```

```
t4= t1 + t5 ;
```

Propagation de copie impossible

La propagation de **t1 = t2**; est inutile car t1 est écrasé immédiatement après avec t1 = t3;.

- Si on remplaçait t1 par t2 dans t4 = t1 + t5;, cela donnerait t4 = t2 + t5;, ce qui serait incorrect, car t1 a été modifié avant d'être utilisé.
- La bonne valeur de t1 lors de son utilisation est t3, donc seule la **dernière affectation est pertinente**.

2. Propagation d'expression

Principe:

Si une variable stocke une expression mais **n'est utilisée qu'une seule fois**, alors il est inutile de la garder. On peut directement remplacer son utilisation par l'expression qu'elle contenait.

Exemple:

Avant optimisation:

```
t1= expr;
```

```
t3= 4* t1 ;
```

Après optimisation:

```
t3= 4* expr ;
```

Remarque:

Si la même expression est utilisée plusieurs fois, alors il vaut mieux garder la variable pour éviter de recalculer plusieurs fois.

3. Élimination des expressions redondantes (ou communes)

Principe:

- Cette technique consiste à éviter de recalculer une expression **si** elle a déjà été évaluée auparavant et que **ses variables n'ont pas changé**. Au lieu de recalculer, on réutilise le résultat stocké dans une variable temporaire.

Autrement dit,

- Si une expression E apparaît plusieurs fois et que les variables qui la composent n'ont pas été modifiées, on remplace ses occurrences suivantes par la variable qui stockait E.

Exemple:

Avant optimisation:

t1 = a + b;

t2 = c * d;

t3 = a + b; // Redondant, car déjà calculé dans t1

t4 = t3 * e;

Après optimisation:

t1 = a + b;

t2 = c * d;

t4 = **t1** * e; // Remplacement de t3 par t1

- a + b est recalculé une deuxième fois (t3 = a + b;), ce qui est inutile.

Donc, on évite un calcul en réutilisant t1 au lieu de recalculer a + b.

Remarque:

Si l'une des variables de l'expression change entre deux utilisations, la valeur recalculée peut être différente et l'optimisation ne peut pas être appliquée.

4. Simplification algébrique

Principe:

- Elle consiste à **remplacer** certaines opérations coûteuses par des opérations équivalentes mais plus efficaces.

1. Optimisation des multiplications

- La multiplication est **plus coûteuse** que l'addition, quelque soit la machine.
- Si une des opérandes est une constante, on peut remplacer la multiplication par une addition.

Exemple:

Avant optimisation:

$Y = Y * 2;$

Après optimisation:

$Y = Y + Y;$

2. Élimination des calculs inutiles

- Si une opération n'a aucun effet sur le résultat final, elle est supprimée.

Exemple:

Avant optimisation:

$X = Y + 1 - 1;$

Après optimisation:

$X = Y;$

Avant optimisation:

$X = Y * 1;$

Après optimisation:

$X = Y;$

Avant optimisation:

$X = Y + 0 ;$

Après optimisation:

$X = Y;$

5. Élimination de code inutile

Principe:

- Elle consiste à éliminer les calculs et les instructions qui n'ont aucun impact sur le résultat final du programme, comme :

- **Calculs dont le résultat n'est jamais utilisé:**

Si une variable est assignée mais jamais utilisée après, l'opération qui la produit est inutile et peut être éliminé.

Exemple:

Avant optimisation:

x=t3;

a[t2]=t5;

a[t4]=t3;

//x est inutile

Après optimisation:

a[t2]=t5;

a[t4]=t3;

Exemple:

Optimisez le code dans la portion de programme suivante:

```
t6 = 4*j  
t8 = j-1  
t9 = 4 * t8  
temp = A[t9]  
t10 = j+1  
t11 = t10-1  
t12 = 4 * t11  
t13 = A[t12]  
t14 = j-1  
t15 = 4 * t14  
A[t15] = t13  
t16 = j+1  
t17 = t16-1  
t18 = 4 * t17  
A[t18] = temp
```

Etape 1:

- Propagation d'expression et Simplification algébrique

$t_{11} = t_{10} - 1$

$\Rightarrow t_{11} = j + 1 - 1$ // propagation d'expr

$\Rightarrow t_{11} = j$ // Simplification algébrique

- Propagation de copie

$t_{12} = 4 * t_{11} \Rightarrow t_{12} = 4 * j$

- Élimination de code inutile

pour t_{10} et t_{11}

De meme pour t_{16} , t_{17} et t_{18}

$t_6 = 4 * j$

$t_8 = j - 1$

$t_9 = 4 * t_8$

$temp = A[t_9]$

$t_{10} = j + 1$

$t_{11} = t_{10} - 1$

$t_{12} = 4 * t_{11}$

$t_{13} = A[t_{12}]$

$t_{14} = j - 1$

$t_{15} = 4 * t_{14}$

$A[t_{15}] = t_{13}$

$t_{16} = j + 1$

$t_{17} = t_{16} - 1$

$t_{18} = 4 * t_{17}$

$A[t_{18}] = temp$

$t_6 = 4 * j$

$t_8 = j - 1$

$t_9 = 4 * t_8$

$temp = A[t_9]$

$t_{12} = 4 * j$

$t_{13} = A[t_{12}]$

$t_{14} = j - 1$

$t_{15} = 4 * t_{14}$

$A[t_{15}] = t_{13}$

$t_{18} = 4 * j$

$A[t_{18}] = temp$

Etape 2:

- Élimination d'expressions redondantes:

- $t_{12}=4*j \Rightarrow t_{12}=t_6$

- $t_{14}=j-1 \Rightarrow t_{14}=t_8$

- $t_{18}=4*j \Rightarrow t_{18}=t_6$

- Propagation de copie

$A[t_{12}] \Rightarrow A[t_6]$ et $A[t_{18}] \Rightarrow A[t_6]$

$t_{15}=4*t_{14} \Rightarrow t_{15}=4*t_8$

- Elimination de code inutile

Pour t_{12} , t_{14} et t_{18}

$t_6 = 4*j$

$t_8 = j-1$

$t_9 = 4 * t_8$

$temp = A[t_9]$

$t_{12} = 4 * j$

$t_{13} = A[t_{12}]$

$t_{14} = j-1$

$t_{15} = 4 * t_{14}$

$A[t_{15}] = t_{13}$

$t_{18} = 4 * j$

$A[t_{18}] = temp$

$t_6 = 4*j$

$t_8 = j-1$

$t_9 = 4 * t_8$

$temp = A[t_9]$

$t_{13} = A[t_6]$

$t_{15} = 4 * t_8$

$A[t_{15}] = t_{13}$

$A[t_6] = temp$

Etape 3:

- Élimination d'expressions redondantes
- $t_{15} = 4 * t_8 \Rightarrow t_{15} = t_9$
- Propagation de copie
- $A[t_{15}] \Rightarrow A[t_9]$
- Élimination de code inutile
- t_{15}
- Propagation d'expr
- $t_8 \Rightarrow (j - 1)$

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t13 = A[t6]
t15 = 4 * t8
A[t15] = t13
A[t6] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t13 = A[t6]
A[t9] = t13
A[t6] = temp

```

```

t6 = 4*j
t9 = 4 * (j-1)
temp = A[t9]
t13 = A[t6]
A[t9] = t13
A[t6] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t10 = j+1
t11 = t10-1
t12 = 4 * t11
t13 = A[t12]
t14 = j-1
t15 = 4 * t14
A[t15] = t13
t16 = j+1
t17 = t16-1
t18 = 4 * t17
A[t18] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t12 = 4 * j
t13 = A[t12]
t14 = j-1
t15 = 4 * t14
A[t15] = t13
t18 = 4 * j
A[t18] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t12 = t6
t13 = A[t6]
t14 = t8
t15 = 4 * t8
A[t15] = t13
t18 = t6
A[t6] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * t8
temp = A[t9]
t13 = A[t6]
A[t9] = t13
A[t6] = temp

```

```

t6 = 4*j
t8 = j-1
t9 = 4 * (j-1)
temp = A[t9]
t13 = A[t6]
A[t9] = t13
A[t6] = temp

```

Implementation

- Comment sauvegarder et mettre à jour l'état des variables et des temporaires?
- Dans quel ordre appliquer les opérations ?
- Combien de fois doit-on repasser le code ?
- implémentation optimale ?